StockSense

Team roles and tasks: who does what

StockSense is a Python inventory system. It tracks stock, warns the owner when to reorder, and shows which products sell well or not at all. We use Python, SQLite and Streamlit, all in one GitHub repo.

Person	Job in one line	Main files
Person 1	Database and core logic	backend/database.py, backend/inventory.py, backend/auth.py
Person 2	Analytics: reorder, ABC, dead stock	backend/analytics.py, tests/test_analytics.py
Person 3	Streamlit screens	app.py, pages/
Person 4	Tests, data, report, slides	tests/, data/, backend/reports.py, docs/

Rules for everyone

- Never push to **main**. Work on your own branch (feature/analytics, feature/ui, ...), then open a pull request.
- Another teammate reviews each pull request before it is merged.
- Run **git pull origin main** every day.
- Comment your code. Be ready to explain ANY part of the project in Q&A.
- Everyone speaks in the presentation.

Person 1: Database and Core Logic

Your files: backend/database.py, backend/inventory.py, backend/auth.py
Goal: Build the foundation: where data is stored and how stock changes.

What you need to do

- Create the GitHub repo and add everyone (DONE: project skeleton is ready).
- Design the database tables: users, suppliers, products, transactions (DONE).
- Write add / edit / delete / search for products and suppliers (DONE).
- Write record_sale and record_purchase so stock updates automatically (DONE).
- Write login (admin / staff) (DONE).
- Review and merge teammates' pull requests; fix merge problems.

What you present

Problem statement, system architecture, database design.

Person 2: Analytics Engine

Your files: backend/analytics.py, tests/test_analytics.py
Goal: Make the system smart: tell the owner what to reorder and what is not selling.

What you need to do

- average_daily_sales: units sold per day over the last 30 days.
- calculate_reorder_point: average daily sales x lead time + safety stock.
- get_reorder_list: products below their reorder point, most urgent first (use heapq / priority queue).
- abc_analysis: sort products by revenue; label A (top 80%), B (next 15%), C (rest).
- dead_stock: products in stock with no sale in the last N days.
- moving_average: 7-day moving average of daily sales for one product.
- Write pytest tests for each function with small hand-calculated examples.

What you present

Algorithms and data structures used (heap, sorting, hash map) with the formulas.

Person 3: Frontend (Streamlit UI)

Your files: app.py, pages/
Goal: Make everything usable through clean screens.

What you need to do

- Dashboard: cards (total products, stock value, today's sales) and a Reorder Now list.
- Products page: table with search, plus forms to add / edit / delete.
- Sales and Purchases page: pick product by SKU, enter quantity, submit; show friendly errors (e.g. not enough stock).
- Analytics page: charts for sales trend, ABC classes, dead stock table.
- Hide admin-only actions from staff users.
- Use the backend functions listed in README.md; do not write SQL in the UI.

What you present

Live demo and user flow.

Person 4: Testing, Data, Reports and Documents

Your files: tests/, data/, backend/reports.py, docs/
Goal: Prove it works, fill it with data, and write everything up.

What you need to do

- Make sample data: ~20 products and a few hundred sales over 60 days (load it with record_sale(..., timestamp=...)).
- Write extra pytest tests for edge cases (empty database, zero stock, bad input).
- CSV export for inventory and sales (PDF export if time allows).
- Write the project report (max 10 pages): problem, scope, tech stack, implementation, testing and results, industry use, challenges, future scope.
- Build the presentation slides (clean, few words, no clutter).
- Keep a record of test results and screenshots for the report.

What you present

Testing and results, industry relevance, conclusion and future scope.

Timeline (presentations start 5 Nov)

Date	What happens
29 Oct	Repo set up, everyone clones it, read README.md
30 Oct	P2 reorder + ABC | P3 login and layout | P4 sample data
31 Oct	P2 dead stock + moving average | P3 product and sales pages | P4 tests
1 Nov	Connect UI to backend, fix bugs together
2 Nov	Alerts, charts, dashboard polish, exports. P4 drafts report
3 Nov	Feature freeze. Full testing. Report and slides finished
4 Nov	Rehearse demo twice, assign speaking parts, record a backup video
5 Nov	Presentation